



RS6x_7x HIF 开发 快速入门

文档页数：27

文档版本号：1.2

发布时间：2026-09-03

版本历史

版本	版本描述	日期
1.2	格式修改。	2026-09-03
1.1	格式修改。	2026-08-14
1.0	文档初版。	2026-06-30

目录

版本历史	ii
目录	iii
图片目录	v
表格目录	vi
1 简介	7
1.1 功能特性	7
1.2 开发步骤	8
2 开发板体验	9
3 开发评估	10
3.1 各平台重点评估项	11
3.1.1 RTOS 平台	12
3.1.2 Linux 平台	12
3.2 资源与时序评估	12
3.2.1 时序评估	12
3.2.1.1 Point Cloud 评估	12
3.2.1.2 Data Cube 评估	14
3.2.1.3 Other Data 评估	15
3.2.2 Memory 资源评估	16
3.2.2.1 Device Memory	16
3.2.2.2 Host Memory	17
3.2.3 IO 资源	17
4 开发调试	18
4.1 基础功能适配	18
4.1.1 Host 移植	18
4.1.2 移植调试	18
4.1.2.1 IO 通知流程	20
4.1.2.2 数据发送流程	20
4.1.2.3 数据接收流程	21
4.2 进阶功能适配	21
4.2.1 SPI 传输性能测试	21
4.2.2 烧录调试	22
4.2.3 多从机调试	22
4.2.4 Sample 调试	22
5 附录-A HIF 雷达上报时序	23

5.1 点云数据上报 23

 5.1.1 单帧处理 23

 5.1.2 多帧处理 24

5.2 Data Cube 数据上报 24

 5.2.1 单帧上报和处理 24

 5.2.2 单帧上报和多帧处理 25

 5.2.3 多帧上报和处理 25

 5.2.4 其他信息上报 25

6 附录-B Host 实时性参数 26

图片目录

图 1-1	HIF 开发步骤	8
图 3-1	点云传输参数配置	13
图 3-2	Data Cube 传输参数配置	14
图 3-3	Other Data 传输参数配置	15
图 3-4	Device Memory	16
图 3-5	Host Memory	17
图 3-6	IO 资源	17
图 4-1	开发调试步骤	18
图 4-2	Port sample UART 传输流程	19
图 4-3	Port sample SPI 或 I2C 传输流程	19
图 4-4	IO 通知流程	20
图 4-5	数据发送通知流程	20
图 4-6	数据接收通知流程	21
图 4-7	Host 实时性参数	22
图 5-1	雷达帧周期示意图	23
图 5-2	点云单帧上报示意图	23
图 5-3	点云多帧上报示意图	24
图 5-4	Data Cube 单帧上报和处理示意图	25
图 5-5	Data Cube 单帧上报多帧处理示意图	25
图 5-6	Data Cube 多帧上报处理示意图	25
图 6-1	Host 实时性参数示意图	26

表格目录

表 1-1	HIF 功能特性表.....	7
表 2-1	基础 sample 介绍.....	9
表 3-1	典型应用参考数据	10
表 6-1	Host 实时性参数.....	26

1 简介

HIF 即 Host Interface，是主控和设备间的数据传输协议。当雷达芯片作为外设（Device）时，用于主控（Host）与雷达芯片间的数据通信，包括命令配置、数据上报、固件烧录等功能。

本文档用于指引软件工程师在方案中快速使用 HIF。HIF 详细信息请查阅 [《RS6x_7x_HIF_参考手册》](#)

1.1 功能特性

HIF 支持两种规格版本，默认使用“Normal”，特性如下表：

表 1-1 HIF 功能特性表

Item			Mini	Normal
场景	应用场景		- 算法和应用在 Device 端开发，且不做大数据量上报 - 低功耗应用开发	- 算法和应用在 Device 端开发，且不做大数据量上报 - 大数据量上报（1D/2D/ADC/高密度点云） - 应用算法在 Host 开发
	数据类型		- 点云上报 - 应用结果上报 - 应用或调试信息上报	- Data Cube 上报 - 点云上报 - 应用结果上报 - 应用或调试信息上报
基础	通信	UART	115200 460800 921600 1000000 2000000	115200 460800 921600 1000000 2000000
		SPI	- Std-SPI <= 8MHz	- Std-SPI - Dual-SPI - Quad-SPI <= 64Mhz
		I2C	100KHz 400KHz	100KHz 400KHz
	基础功能	并发	支持	支持
		阻塞上报	支持	支持

Item			Mini	Normal
		非阻塞上报	支持	支持
		消息校验	支持	支持
		扩展头	不支持	支持（当前版本未支持）
		PM	支持	支持
		使用 DMA 上报	不支持	支持
		0 拷贝	支持	支持
	存储资源	线程栈（Bytes）	512	1024
		上报队列	- Frame Node: 4 - Data Node: 16 - Semaphore Node: 2	- Frame Node: 16 - Data Node: 64 - Semaphore Node: 4
		命令和回复 buff (Bytes)	256	512
扩展	烧录		支持	支持
	OTA		不支持	支持（当前版本未支持）
	多从机		支持	支持
	应用层分片		不支持	支持（默认关闭）
	应用层重传		不支持	支持（默认关闭）
	传输层分片		不支持	支持
	传输层重传		不支持	支持（默认关闭）
	加解密		不支持	支持（默认关闭）
	XIP/SRAM 运行		SRAM (可选 XIP)	SRAM (可选 XIP)

1.2 开发步骤

PSIC 的雷达支持免开发和 SDK 开发两种开发模式。用户在雷达应用中使用 HIF，可按照如下流程进行：

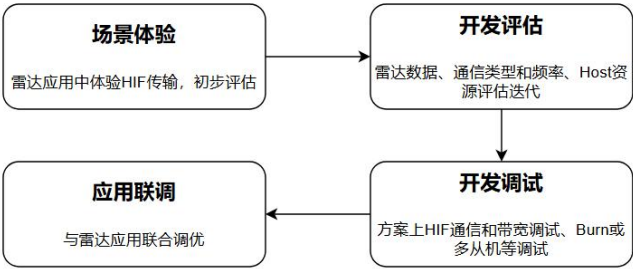


图 1-1 HIF 开发步骤

2 开发板体验

正和微芯为雷达应用数据（移动点云、微动点云、1D Data-Cube、2D Data-Cube）分别提供了体验 Sample，用户可使用开发板和 PC Tools 对雷达应用和 HIF 传输进行体验和评估。

此过程可助用户达成如下目的：

1. 对 HIF 通信进行初步评估

不同 Sample 对应不同的 HIF 传输带宽需求，可作为通信类型和速率的评估参考，如：传输 1D Data-Cube 数据，HIF 能选择 SPI 和 UART 不能选择 I2C。

用户还可基于 Sample，通过 Tools 调整雷达参数和 HIF 通信类型、频率，立即可观察是否有 Checksum 错误和丢帧等提示来判断传输带宽是否足够。

2. 了解 HIF 通信协议，可作为移植调试参考

通过 Tools 的 Log， 对照文档了解 HIF 的帧格式和交互流程。在后续的开发移植时，可对比 Debug。

表 2-1 基础 sample 介绍

Sample Name	Report Data Type	Default Parameter	Communication
Tracking	移动点云	帧周期：50ms 移动点数：max 9KB/帧	SPI: Standard, 60MHz (默认) UART: 1000000bps (可选) I2C: 400KHz (可选，雷达点云点数<128)
Posture	移动点云、 微动点云	帧周期：50ms 移动点云：max 9KB/帧 微动点云：max 3KB/帧	SPI: Standard, 60MHz (默认) UART: 1000000bps (可选)
Sleep	移动点云、 微动点云、 Interval0 1D Data Cube	帧周期：50ms 1D Data Cube：8KB/帧 移动点云：max 9KB/帧 微动点云：max 3KB/帧	SPI: Standard, 60MHz (默认) UART: 1000000bps (可选)
ReportDataCube1D	1D Data Cube	帧周期：5ms 1D Data Cube：64KB/8 帧	SPI: Standard, 64MHz (默认) UART: 1000000bps (可选)
ReportDataCube2D	2D Data Cube	帧周期：100ms 2D Data Cube：256KB/帧	SPI: Standard/Dual/Quad, 64MHz (默认)

Sample 详情请参考 [《RS6x_7x_毫米波雷达_Sample_参考手册》](#)。

3 开发评估

在“场景体验”阶段，用户可通过 RadarDebugTool 进行雷达体验，初步确定雷达参数和 HIF 通信类型。本章介绍用户在评估时需重点关注哪些参数，及怎么在“通信类型和速度”、“传输策略”和“雷达参数”之间权衡调整，迭代出最佳方案。

算法是雷达开发的核心，按雷达算法所在设备可分为：算法在雷达设备端和算法在主控端。算法在雷达设备端时，上报给主控的数据通常为算法处理后的结果，上报数据量较小，通信几乎不影响算法开发，如下图的人存应用；算法在主控端时，上报数据量较大，通信将显著地占据帧周期内部分时间，此时 HIF 通信是雷达设计的重要约束，下文将围绕算法在主控端进行介绍。

下表是典型应用下，需重点关注的参数数据，供设计参考。

表 3-1 典型应用参考数据

Radar Solution	Data Type	Radar Param	Communication		Host		
			Types	Trans Time (ms)	CPU Max Calc Time (ms)	Max response latency (ms)	SramSize (KB)
Ultra-Low-Power Human Presence	Presence Detection	- Report state - Period (ms): 1000 - TxDuration (ms): 0.2	UART 115200	0.1	1000	>= 1000	0
			I2C 100KHz	0.1	1000	>= 1000	0
Tracking	Point Cloud	- Motion Cloud: 256 - Present Cloud: 128 - Period (ms): 50 - TxDuration (ms): 16.5	UART 2MHz	23.22	26.78	>= 1000	9
			SPI-Std 64MHz	1.21	48.79	>= 1000	9
	Point Cloud	- Motion Cloud: 512 - Present Cloud: 256 - Period (ms): 50 - TxDuration (ms): 16.5	SPI-Std 64MHz	2.24	47.76	>= 1000	18
	Data Cube	- Data Cube (KB): 256 - mmw Period (ms): 100 - TxDuration (ms): 16.5	SPI-Std 64MHz	43.26	56.74	40.24	256
			SPI-Dual 64MHz	26.78	73.22	56.72	256
	Data Cube	- Data Cube (KB): 512 - mmw Period (ms): 100 - TxDuration (ms): 16.5	SPI-Dual 64MHz	52.89	47.11	30.61	512
Posture	Point Cloud	- Motion Cloud: 768 - Present Cloud: 256 - Period (ms): 50 - TxDuration (ms): 16.5	SPI-Std 64MHz	2.78	47.22	47.22	24
	Point Cloud	- Data Cube (KB): 256 - mmw Period (ms): 50 - TxDuration (ms): 16.5	SPI-Dual 64MHz	26.78	49.42	6.72	512
			SPI-Quad 64MHz	18.59	49.37	14.91	512
	Data Cube	- Data Cube (KB): 512 - mmw Period (ms): 50 - TxDuration (ms): 16.5	SPI-Quad 64MHz	36.42	-2.92	-2.92	1024
	Point Cloud	- Motion Cloud: 768 - Present Cloud: 256 - Period (ms): 75 - TxDuration (ms): 36	SPI-Std 64MHz	2.78	72.22	72.22	24
Vital Monitoring	Data Cube	- Data Cube (KB): 1 - mmw Period (ms): 10 - TxDuration (ms): 0.2	UART 2MHz	5.23	4.77	>= 1000	2
	Cube	- Data Cube (KB): 8 - mmw Period (ms): 10 - TxDuration (ms): 0.2	SPI-Std 64MHz	1.56	8.44	158.44	16
Sleep Monitoring	Data Cube	- Data Cube (KB): 64 - mmw Period (ms): 20 - TxDuration (ms): 13.68	SPI-Std 64MHz	11.4	16.84	28.60	128
	Point Cloud + Cube	- Motion Cloud: 256 - Data Cube (KB): 8 - mmw Period (ms): 20 - TxDuration (ms): 13.68	SPI-Std 64MHz	2.72	17.28	>= 1000	22

如上表，各重点参数对性能有不同的影响，为便于理解，用户可阅读“附录-A HIF 雷达上报时序”和“附录-B Host 实时性参数”了解传输模型，各参数影响简述如下：

1. Max Calc time (Host 计算可用时间)

Host 接收 Device 的雷达数据用于计算（算法处理），在应用场景和 Host 平台确定的情况下，可用计算时间是决定总算力是否满足应用的关键因素。

若以雷达帧周期为参考时间，Host 缓存单倍 buffer 时，计算时间为帧周期减去传输时间；Host 缓存多倍 buffer 时，计算时间为整个帧周期，但传输期间参与的计算需“打折扣”（表中输入参数：Host CPU utilization rate），与 Host 采取的策略有关，如：多核使用独立核接收则全部时间可用于计算，单核使用 DMA 接收则大部分时间可用于计算，单核中断或轮询接收则基本上无时间用于计算。此外，对于多任务的 Host，需预留一定的计算时间用于处理其他紧急事务且不影响当前帧数据的计算。

2. Max response latency (Host 响应延迟)

类似 Linux 平台的 Host，非实时 OS 调度策略会导致不能及时响应读取雷达数据或读取雷达数据中途调度出去，如 Device 拉起通知 IO 至 Host 开始进行数据传输的时间通常需要十几毫秒至几十毫秒。对于高帧率的雷达应用，若不能缓存多帧数据传输，可能存在雷达数据还未传输完就被下一帧数据覆盖情况。因此，响应延迟是类 Linux 的 Host 的重要评估参数，影响传输策略及传输 Buffer。

3. Host Cache frames(Host 缓存数据 Buffer)

对于算力不均匀的 Host（计算任务频繁被高优先级任务抢占），可缓存多帧数据应对。对于响应延迟大的 Host，Device 需支持缓存多帧传输，Host 也需要开辟对应的 Buffer 接收和计算。对于高帧率、少数据的雷达应用，Device 也可缓存几帧一起传输，Host 开辟相应 Buff 处理。

4. Common Param（通信类型和频率）

此参数直接影响方案可行性，评估时通常以传输时间衡量。它影响是否需要缓存多帧传输及 Host、Device 需要的传输计算 Buffer 大小，评估时参考上表依据数据类型初选择，再根据参数细调。Host 具有的通信类型和支持的频率及 IO 也是影响通信选择的决定性因素。

评估时，依托《Radar_HIF_Calculation》表格进行参数细调，主要步骤如下：

1. 确定雷达参数和上报数据类型
2. 使用《Radar_HIF_Calculation》的“Point Cloud”、“Data Cube”或“Other Data”页进行评估计算，获得重要参数数据。
3. 使用《Radar_HIF_Calculation》的“Memory”和“IO”页进行评估计算，获得需要的硬件资源，对比 Host 的实际资源进行判定是否可用。
4. 重复步骤 1 至 3，获得最佳方案。

3.1 各平台重点评估项

RTOS Host 和 Linux Host 其特性差异较大，因此对资源评估的关注点不同。

3.1.1 RTOS 平台

RTOS Host 的特点：

1. 响应快
2. SRAM 资源少（通常运行在 MCU 上）
3. 算力较低

需重点关注的参数：

1. Host CPU 的 Max Calc time（Host 可用于计算的时间）
2. Host SRAM 资源

3.1.2 Linux 平台

Linux Host 的特点：

1. 响应慢
2. SRAM 资源多
3. 算力高
4. 可能存在算力不均匀（CPU 不能持续用于雷达计算，可能切出处理其他事情）

需重点关注的参数：

1. Host 的 Max response latency（最大响应延迟）
2. 算力均匀，即 Host 缓存多帧数据做缓冲

3.2 资源与时序评估

3.2.1 时序评估

以下按照不同的数据类型介绍传输时间、Host 可用计算时间、Host 响应延迟等时序相关的评估。

3.2.1.1 Point Cloud 评估

点云评估计算界面如下图，蓝色为输入参数，橙色为输出参数。

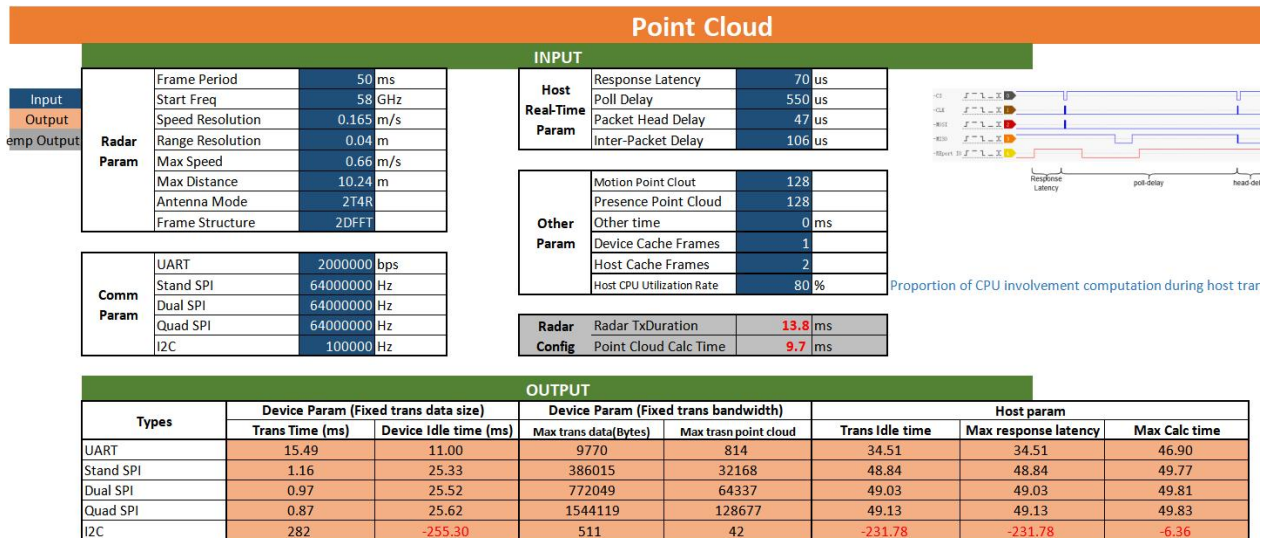


图 3-1 点云传输参数配置

输入参数：

1. Radar Param (雷达参数)：依据雷达配置填写，例如 来自 RadarDebugTool
2. Comm Param (通信参数)：根据 Host 支持的通信方式和频率填写，可同时评估多种通信方式。
3. Host Real-Time Param (Host 实时性参数)：若无精确数据时可根据附录-A 的数据填写，或可通过“port_device_time_meas”工程进行测量（详细参考 4.2.1 SPI 传输性能测试）。
4. Other Param (其他参数)：
 - + Other Time：可用于预留时间的输入，例如：每帧预留 5ms 用于高优先级事务处理。
 - + Device Cache Frames：Device 端可缓存多少帧数据上报，若为 1 表示当前帧需上报完数据，若为 2 表示可缓存 2 帧数据上报，影响 Max response time。
 - + Host Cache Frames：Host 可缓存多少帧数据处理，解决 Host 算力不均匀问题。
 - + Host CPU utilization rate：Host 传输期间 CPU 参与雷达计算的比例，与 Host 接收的策略有关。

输出数据：

1. Trans Time（传输耗时）：评估帧周期内数据是否有足够时间完成传输。
 - 若偏差为正表示有足够时间完成传输，值越大余量越多，也可降低传输速度调整；
 - 若偏差为负表示传输时间大于帧周期，可调高传输速率看是否转正，否则此通信方式不可用。
2. Device Idle Time(radar 设备空闲时间)：帧周期减去硬件收发波时间核传输数据时间，对于低功耗应用用于评估可休眠时间。
3. Max trans data(最大传输数据)和 Max trans point cloud(最大传输点云个数)：用于评估 Host 支持的最高传输带宽下，可以支持传输的雷达数据量和参数。
4. Trans Idle time (Host 传输空闲时间)：帧周期减去传输时间，用于评估 Host 可全力用于雷达计算的时间。
5. Max response latency（最大响应延迟）：Device 拉高 IO 至 Host 读取数据的时间。此项受 Device Cache Frame 即 Device 的缓存大小影响。

6. Max Calc time（Host 可用于计算的时间）：结合 Host 的平均算力可用于评估当前应用的计算时间足够。

3.2.1.2 Data Cube 评估

Data Cube 评估计算界面如下图，蓝色为输入参数，橙色为输出参数。

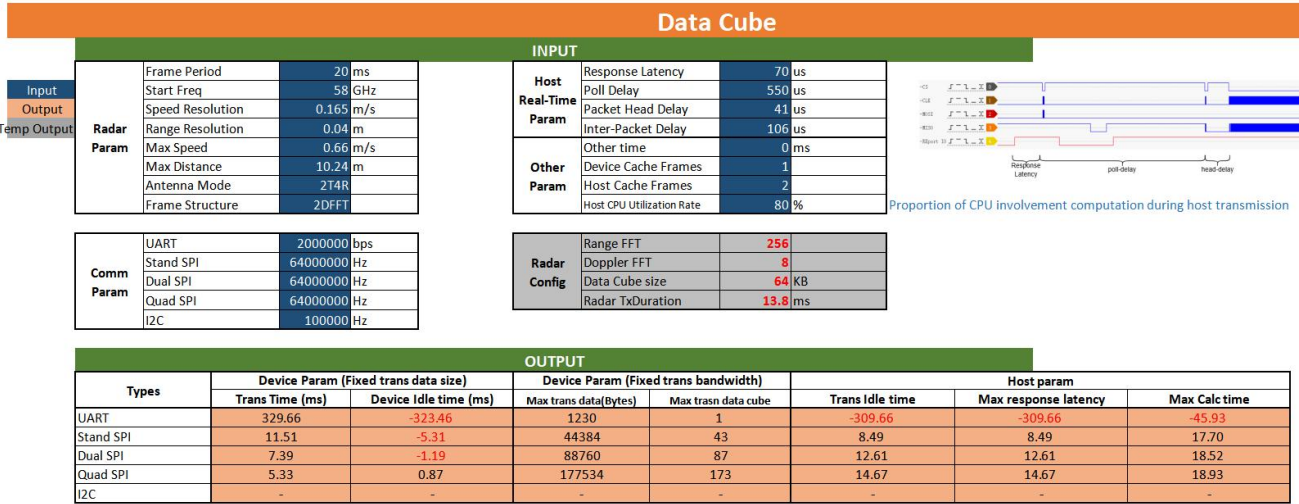


图 3-2 Data Cube 传输参数配置

输入参数：

1. Radar Param（雷达参数）：依据雷达配置填写，例如 来自 RadarDebugTool
2. Comm Param（通信参数）：根据 Host 支持的通信方式和频率填写，可同时评估多种通信方式。
3. Host Real-Time Param（Host 实时性参数）：若无精确数据时可根据附录-A 的数据填写，或可通过“port_device_time_meas”工程进行测量（详细参考 4.2.1 SPI 传输性能测试）。
4. Other Param(其他参数):
 - + Other Time：可用于预留时间的输入，例如：每帧预留 5ms 用于高优先级事务处理。
 - + Device Cache Frames： Device 端可缓存多少帧数据上报，若为 1 表示当前帧需上报完数据，若为 2 表示可缓存 2 帧数据上报，影响 Max response time。
 - +Host Cache Frames： Host 可缓存多少帧数据处理，解决 Host 算力不均匀问题。
 - + Host CPU utilization rate： Host 传输期间 CPU 参与雷达计算的比例，与 Host 接收的策略有关。

输出数据：

1. Trans Time（传输耗时）：评估帧周期内数据是否有足够时间完成传输。
 - 若偏差为正表示有足够时间完成传输，值越大余量越多，也可降低传输速度调整；
 - 若偏差为负表示传输时间大于帧周期，可调高传输速率看是否转正，否则此通信方式不可用。
2. Device Idle Time(radar 设备空闲时间): 帧周期减去硬件收发波时间核传输数据时间，对于低功耗应用用于评估可休眠时间。
3. Max trans data(最大传输数据)和 Max trans data cube(最大传输 Data Cube): 用于评估 Host 支持的最高传输带宽下，可以支持传输的雷达数据量和参数。

- 2. Device Idle Time(radar 设备空闲时间): 帧周期减去硬件收发波时间核传输数据时间, 对于低功耗应用用于评估可休眠时间。
- 3. Max trans data(最大传输数据): 用于评估 Host 支持的最高传输带宽下, 可以支持传输的雷达数据量和参数。

3.2.2 Memory 资源评估

《Radar_HIF_Calculation》的 memory 页, 提供了快速获得 HIF 所需 Flash (静态存储或 XIP) 和 SRAM 的大小的表格。

3.2.2.1 Device Memory

如下图, 用户可配置 “Input” 选项, 即可获得最终 (Total) Memory 大小。

- 1. “Base” 的 DLL 是必选项, PHY 可选其中一种通信方式 (此通信方式可依据后续章节 “参数设计” 确认) ;
 - 2. “Extent” 为扩展功能, 其中分片 (Split) 默认开启; 应用层重传和传输层重传参考文档《HIF 开发手册》使用, 默认不开启;
 - 3. “App” 表示开启应用需要增加的传输缓存 buffer ;
- (1) Data Cube Size 表示要传输的数据量大小, 可为 1/2/4/8/16/32/64/128/256/512 KB, 可从 Data Cube 评估的表格页获得或者可根据文档 [《RS6x_7x_毫米波雷达评估_快速入门》](#) 计算获得。Report Period 表示上报周期, 若为 1 表示每帧及时上报, 若为 2 表示可缓存 2 帧一起上报, 如此类推, 与 Host 响应延迟及帧周期有关 (详情 3.2.1.1) , 也对应时序评估中的 “Device Cached frames” ;
- (2) Point Cloud 表示要传输的各类点云个数 (详情参考 3.2.1.1)。

Device Memory Size					Flash (KB)	SRAM (KB)
Base	DLL	Command/Response length(Bytes)		512	10.64	11.61
		HIF thread stack(Bytes)		1024		
		Report data node counter		64		
		Report frame node counter		16		
		Block semaphore counter		4		
	PHY	SPI		Y	0.34	0.44
		UART		N	0	0
		I2C		N	0	0
Exten	Split	Split node counter	4	Y	3.03	1.45
	Retry	Application layer retry		N	0	0
		Transfer layer retry		N	0	0
	PM	Power Manage		N	0	0
	App	DataCube	Data Cube Size(KB)	32	N	0
Device cache frames			1			
PointCloud		motion	768	N	0	0.00
		presence	256			
Total					14.01	13.50

图 3-4 Device Memory

3.2.2.2 Host Memory

Host Driver Flash 和 SRAM 在不同规格下差异较少，作为基础 Size。应用占用 SRAM 是主要评估项，与应用数据大小和缓存帧数有关。

1. 应用数据为 Data Cube。例如 Cube Size 为 64KB,缓存 2 帧数据,那么需要应用 SRAM 为 64KB*2=128KB
2. 应用数据为点云。例如 motion cloud 768, presence cloud 256, 缓存 2 帧, 需要 SRAM buffer = (768 + 256) * 12 * 2 B = 24KB 。

注意：缓存帧数“Host Cache frames”需与时序评估中的“Host Cache frames”对应。在时序评估中，它影响 Host 算力均匀问题的解决。

Host Memory Size				Flash (KB)	SRAM (KB)
Driver	HIF host driver			17	4.5
App	DataCube	Data Cube Size(KB)	32	N	
		Cached frames	1		
	PointCloud	motion Input →	768	N	
		presence	256		
		Host Cached frames	2		
Total				17.00	4.50

图 3-5 Host Memory

3.2.3 IO 资源

《Radar_HIF_Calculation》的 IO 页，提供了 IO 配置的表格。如下图：

1. 只需选择“通信方式”和“打印输出”
 - (1) 通信方式可依据后续“3.2.1 时序”章节获得
 - (2) 打印输出可用于 Device 调试，可以以测试点的方式存在，即可不连接 Host。
2. 自动匹配的 IO 会以绿色的方式表示
 - (1) 此表以 Device 角度展示，Host 需有与之匹配的功能
 - (2) BOOT pin 用户必须连接，用于固件烧录
 - (3) NRST 用于进入烧录模式和复位 Device，若模组独立供电也可以通过断电方式实现。

多从机时，注意哪些 IO 可复用。I2C 通信的 SDA 和 SCL 可复用，SPI 通信时 CS 不可复用，其他 IO 可复用；UART 通信时 Tx 和 Rx 不可复用。SPI 和 I2C 通信时 Report IO 不可复用。

Communication Mode				print Output
SPI-Standard				Y
Type	Func	Pin	IO	Notes
UART-0	TX	PB0	O	
	RX	PB1	I	
UART-1	TX	PA0	O	
	RX	PA1	I	
SPI	CS	PA0	I	When mutple slave device are present, this IO cannot be multiplexed among all slaves.
	CLK	PA1	I	When mutple slave device are present, this IO can be multiplexed among all slaves.
	MOSI/IO0	PA2	IO	When mutple slave device are present, this IO can be multiplexed among all slaves.
	MISO/IO1	PA3	IO	When mutple slave device are present, this IO can be multiplexed among all slaves.
	IO2	PA4	IO	When mutple slave device are present, this IO can be multiplexed among all slaves.
I2C	SDA	PA4	IO	When mutple slave device are present, this IO can be multiplexed among all slaves.
	SCL	PA5	IO	
ReportIO	ReportIO	PA6	O	For host read and notification and host wake-up, Madtory for data reporting over SPI/I2C
NRST	NRST	NRST	I	Reset device
	BOOT	PB0	I	Enter burn mode

图 3-6 IO 资源

4 开发调试

用户在确定参数后，进入 HIF 通信调试，可按照如下步骤进行：

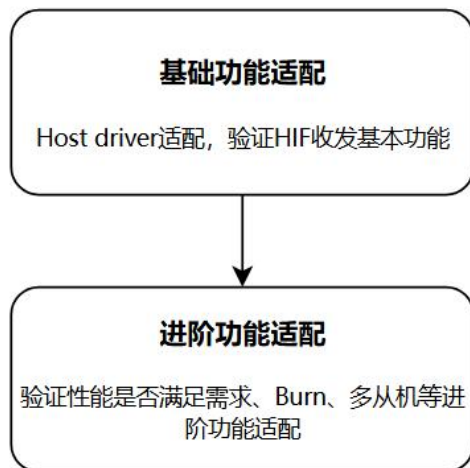


图 4-1 开发调试步骤

4.1 基础功能适配

功能适配目的是移植 Host Driver 并调通与 Device 的通信，按以下流程进行：

1. 移植 Host Driver 至目标平台，编译通过。
2. 烧录 hif_port_device_xxx.img (xxx 为通信方式和参数，如 uart_921600)烧录至 Device。
3. 调试 Host 与 Device 之间的通信。

4.1.1 Host 移植

参考文档 [《RS6x_7x_HIF_参考手册》](#) “5.1 Host Driver 移植” 进行适配移植。

4.1.2 移植调试

Host driver 适配后，host 使用 hif_port_host_sample，device 使用 hif_port_device_sample 进行通信调试。
注：

hif_port_device_sample 请直接使用产品包中的 img，例如：

1. hif_port_device_uart_921600.img : uart 通信，波特率为 921600bps
2. hif_port_device_i2c_3a.img : i2c 通信，软件兼容 100KHz/400KHz，通信地址 0x3A (7bits)
3. hif_port_device_spi_std_64mhz.img : 单线 SPI 通信，最高支持 64MHz 通信频率
4. hif_port_device_spi_dual_64mhz.img : 双线 SPI 通信，最高支持 64MHz 通信频率
5. Hif_port_device_spi_quad_64mhz.img : 四线 SPI 通信，最高支持 64MHz 通信频率

这对 Sample 的流程是：

- 1. 间隔 1S，device 上报一个递增序列号；
- 2. 接着，Host 发送获取版本号的命令并接收回复的版本信息；

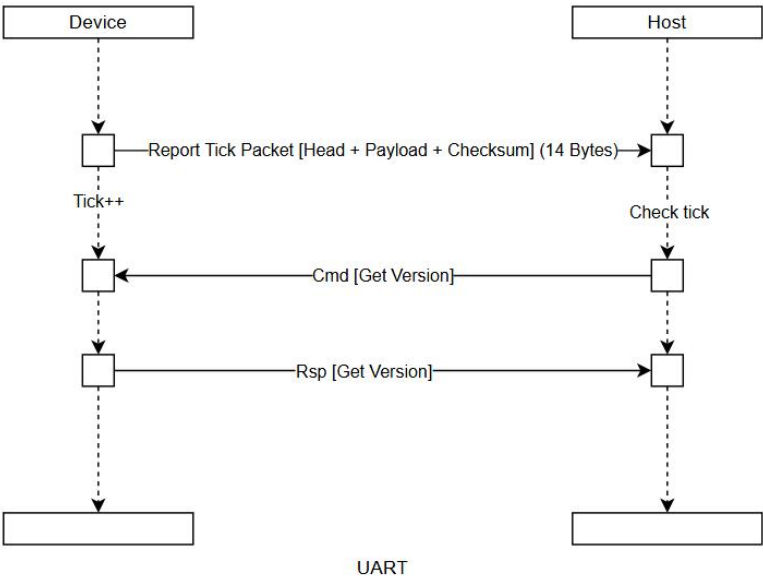


图 4-2 Port sample UART 传输流程

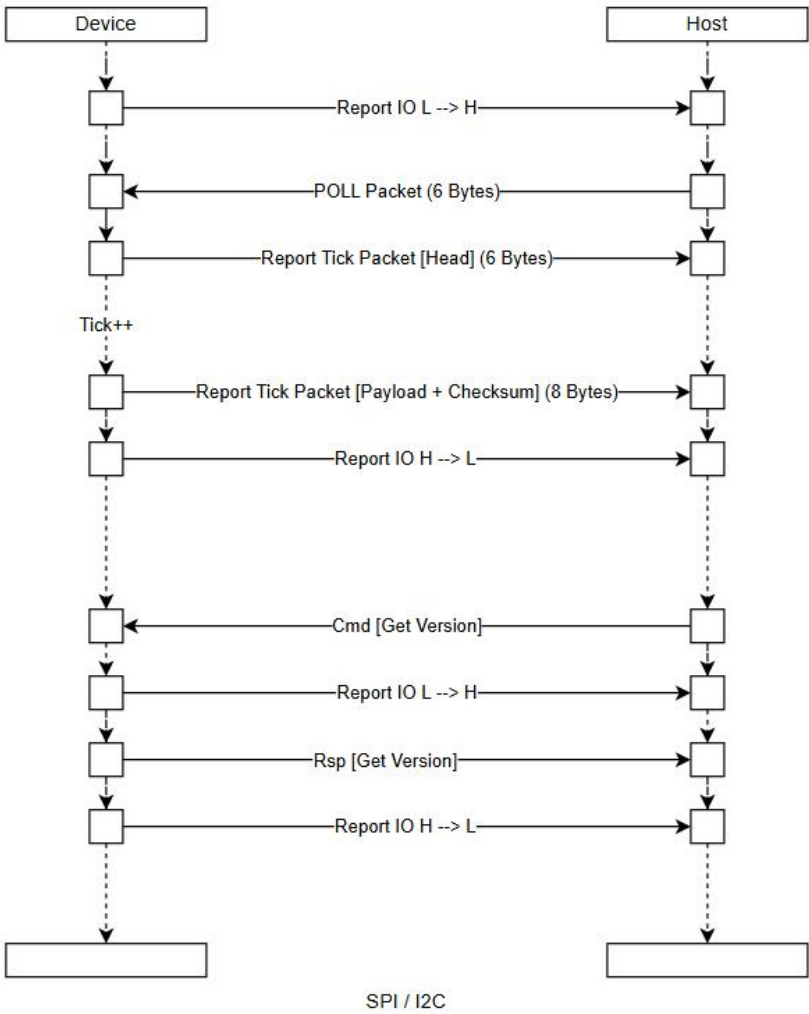


图 4-3 Port sample SPI 或 I2C 传输流程

对于 UART 传输方式，先调试数据接收通路，再调试数据发送通路。对于 SPI/I2C 传输方式，先调试 IO 通知流程，再调试数据发送通路，最后调试数据接收通路。

4.1.2.1 IO通知流程

1. 先确认 port_io 是否有接收到通知，默认使用 GPIO 的上升沿中断触发。
2. 若 port 能收到 IO 触发，再确认 llc_notify_io_irq_handle 接收到触发。
3. 最后确认 HIF_TL_DataNotify_CB 收到触发，至此 IO 通知流程调通。

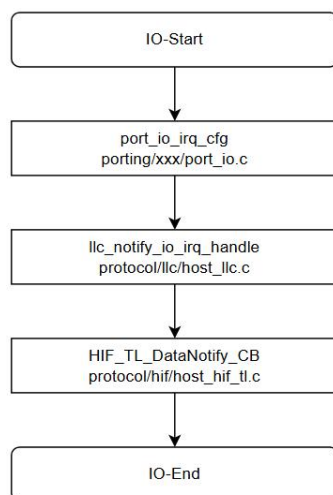


图 4-4 IO 通知流程

4.1.2.2 数据发送流程

1. 先确认传输层 HIF_TL_SendMsg（发送 Get_Version 命令）或 HIF_TL_SendPoll（SPI/I2C 传输发送 poll）有被调用。
2. 确认链路层 LLC_Send 有被执行
3. 确认 Port 层有被调用，若 Device 端还未接收到数据，确认 port 驱动配置是否正确（如：pinmux 是否配置正确，时钟是否开启，可借助逻辑分析仪抓取端口信号确认发送数据）。

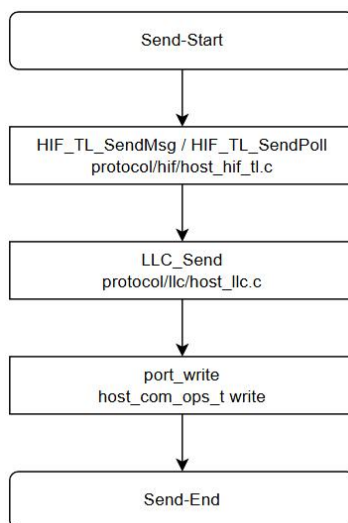


图 4-5 数据发送通知流程

4.1.2.3 数据接收流程

1. 先确认 port 的 read 接口是否有正确接收到数据，若没有检查驱动配置，可通过逻辑分析仪抓取端口信号分析。
2. 再确认链路层和传输层是否正确接收到数据。

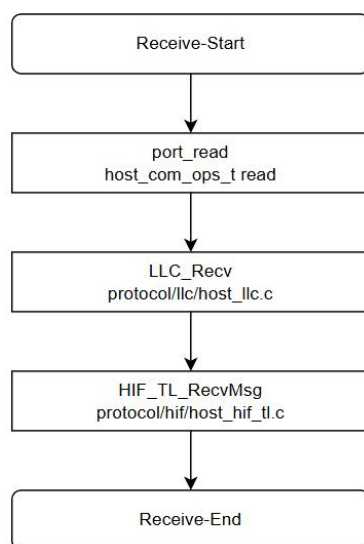


图 4-6 数据接收通知流程

4.2 进阶功能适配

基于上一章节基础通信调通的情况下，继续调试其他基础功能，包括如下内容（可选）：

1. SPI 传输性能和 Host 实时性评估

类似 Data Cube 的传输，对传输带宽要求高及 host 为 Linux 类的方案，可通过此 Sample 实测 host 的实时性参数（如响应延迟），便于精确评估带宽是否足够。

2. 烧录 img 流程

Host 需要通过直接烧录 img 至 Device Flash 的方式进行 Device 固件升级更新。

3. 多从机调试

若 Host 需要同时控制多个 Device，则需要通过此 Sample 验证此功能。

4. Sample 调试

基于基础 Sample（表 2-1 基础 Sample 介绍）的功能验证，可作为用户应用的基础参考。

4.2.1 SPI 传输性能测试

Device 烧录 hif_port_device_time_meas.img，host 运行 hif_port_host_sample，观察 host 的打印输出。

Device 每隔 1S 上报 32KB Data Cube，Device 通过采集 Host 的传输时间算出 SPI 传输频率及 Host 响应延迟等参数。统计一段时间，上报平均值和最大值至 Host，Host 打印输出此内容。

注：仅供参考，用逻辑分析仪更准确。

	type:	avg	max
report-latency:		131	146
poll-delay:		553	554
head-delay:		86	87
inter-packet:		94	94
spi-clock:		16	16

图 4-7 Host 实时性参数

如上图，avg 表示统计平均值，max 表示统计最大值。Report-latency 表示 host 响应延迟，其他参数可对应“附录-B Host 实时性参数”。

4.2.2 烧录调试

Host 烧录使用 burn_sample 验证。需要适配好 Device 的 UPD、RST pin，并适配好获取 img 的接口 port_store.c。

核心流程如下：

```
for (int count = 0; count < HIF_BURN_RETRY_COUNT; count++) {
    do {
        /* host enter burn mode */
        status = Host_BurnMode_Enter(devHandle);
        if (status != HOST_ERRCODE_SUCCESS) {
            HIF_LOG_PRINT("device(%p) enter burn mode fail(%d)\n", devHandle, status);
            break;
        }

        /* host burn image */
        status = Host_Burn_Image(devHandle, "image_user");
        if (status != HOST_ERRCODE_SUCCESS) {
            HIF_LOG_PRINT("device(%p) burn image fail(%d)\n", devHandle, status);
        }

    } while(0);

    HIF_LOG_PRINT("exit burn mode\n");
    /* host exit burn mode */
    Host_BurnMode_Exit(devHandle);
    if (status == HOST_ERRCODE_SUCCESS) {
        break;
    }
}
```

4.2.3 多从机调试

多从机使用 multi_device_sample。可响应不同 Device 的上报。

4.2.4 Sample 调试

Host 运行 cmd_and_report_sample 可配合 Device 5 个基础 Sample 验证数据上报和命令下发。

5 附录-A HIF 雷达上报时序

本章介绍雷达的数据产生、处理与 HIF 上报的处理流程。如下图，Device 端以帧为单位，每帧分三部分：硬件收发波、数据处理及空闲态。Device 与 Host 之间的 HIF 通信及 Host 的数据处理都以此作为时间轴参考。

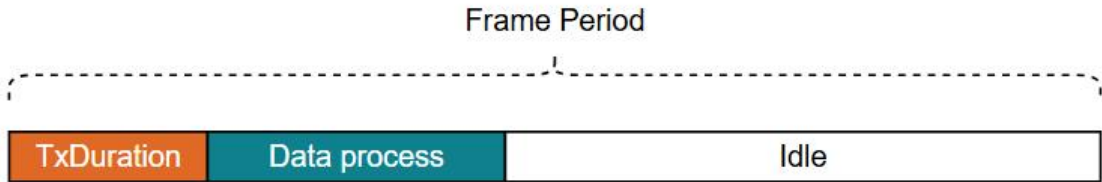


图 5-1 雷达帧周期示意图

5.1 点云数据上报

Device 在“Data processing”阶段将原始 Data Cube 数据加工成点云，再通过 HIF 上报至 Host。此上报可分两种情况：

1. 单帧处理：雷达帧周期较大，Host 实时性较强，可在每帧时间内传输和处理完当前帧数据。
2. 多帧处理：雷达帧周期较小，上报数据较少，可累积多帧数据上报和处理。

5.1.1 单帧处理

这种处理，Device 和 Host 只需 1 倍点云 buff 即可，是最省 SRAM 的方式。但需要在下一帧数据上传前处理完当前帧数据。如下图，Device 上传数据的最大有效时间（Time for transmitting data）为一个帧周期。若 Host 为 Linux 等响应延时（Host response delay）较大的平台，其有效传输时间为 $\text{Time_for_transmitting_data} - \text{Host_response_delay}$ 。Host 端，多倍 Buff 时，通过 HIF 接收数据的时间，其 CPU 用于数据处理的算力根据是否使用 DMA 或多核处理的情况进行折算。

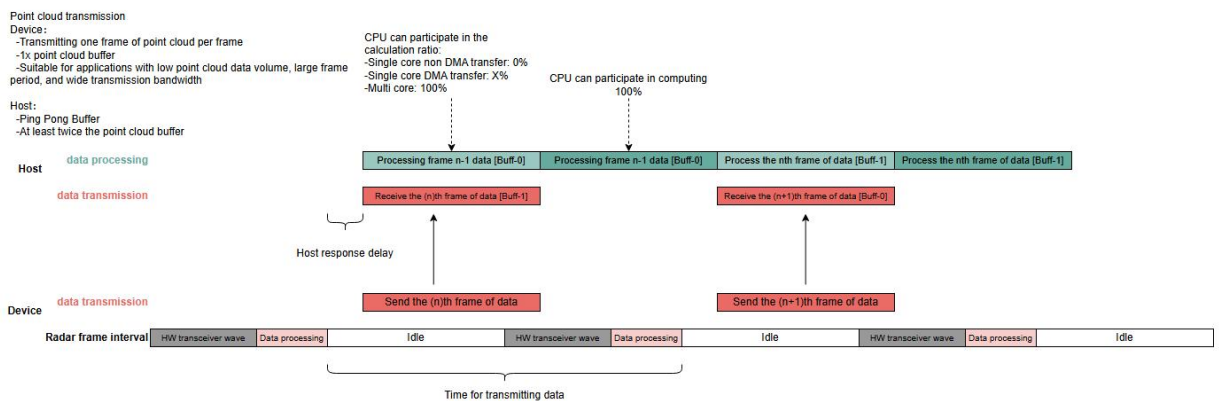


图 5-2 点云单帧上报示意图

5.1.2 多帧处理

多帧上报的方式，Device 多倍 buffer 对 Host 实时性要求变宽松，Host 多倍 Buffer 时能为 Host 提供更多的处理时间。对于帧周期小，数据量少的应用（如帧周期为 5/10ms，点云个数少于 128 个），建议采用此方式。

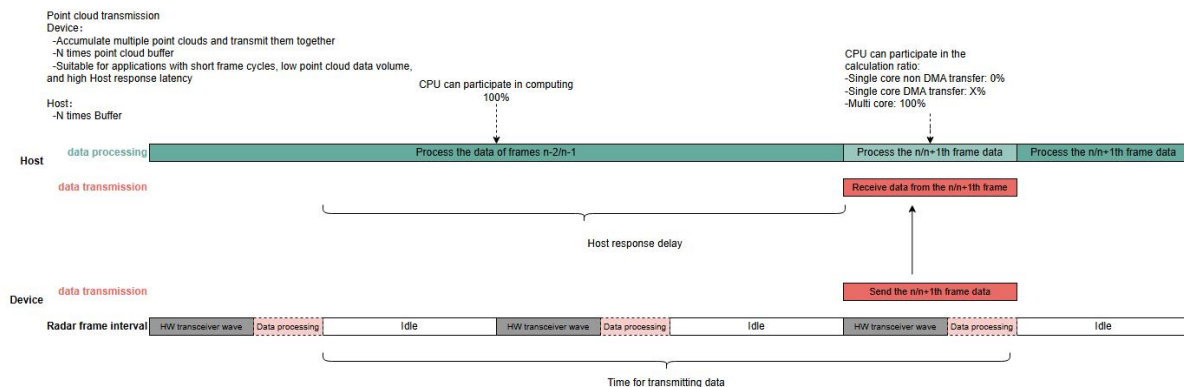


图 5-3 点云多帧上报示意图

5.2 Data Cube 数据上报

Device 在“HW transceiver wave”阶段因与硬件需要互斥，软件不能操作 Data Cube buffer，“Data processing”阶段无（或极少量）数据加工处理，“Idle”阶段可用于复制数据至 HIF Buff 及上报至 Host。此上报可分三种情况：

1. 上报数据量大且雷达帧周期较大的应用，可在每帧时间内传输和处理完当前帧数据。
2. 数据量大且雷达帧周期较小的应用，“Idle”时间内完成传输，Host 可缓存多帧处理
3. 数据量少且雷达帧周期较小的应用或 Host 响应延迟较大的场景，Device 端可缓存多帧一起上报。

5.2.1 单帧上报和处理

HIF 传输数据的时间只有 Idle - Host_response_delay。Host 处理数据的时间不能跨下一帧。适合帧周期较大且 Host 的响应延迟较小的应用。此方式对 Host 的 Device 的 buff 使用最少，Host 只需 1 倍 buffer，Device 使用滑动 buffer 更少。

Suitable for applications with large data volumes and large frame periods, such as:
 -Tracking: Cube 256KB; Frame period of 100ms; wave generation time of 16.5ms
 -Tracking: Cube 512KB; Frame period of 100ms; wave generation time of 16.5ms

Device:
 -<2x Cube buffer
 -The data needs to be processed before the previous frame

Host:
 -1x Cube buffer
 -The data needs to be processed before the next frame

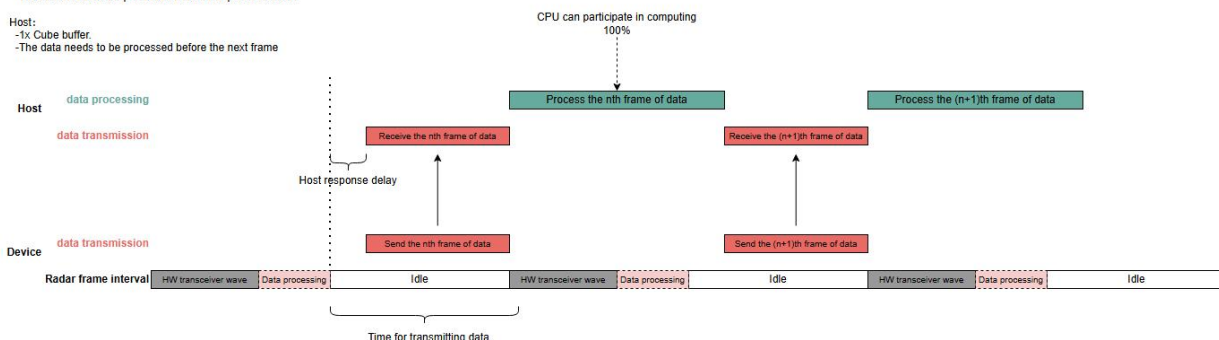


图 5-4 Data Cube 单帧上报和处理示意图

5.2.2 单帧上报和多帧处理

数据传输如“单帧上报和处理”，Host 可缓存多帧数据处理，可应对 Host 算力不均匀问题（不能持续提供高算力，可能被其他更高优先级事务抢占）。

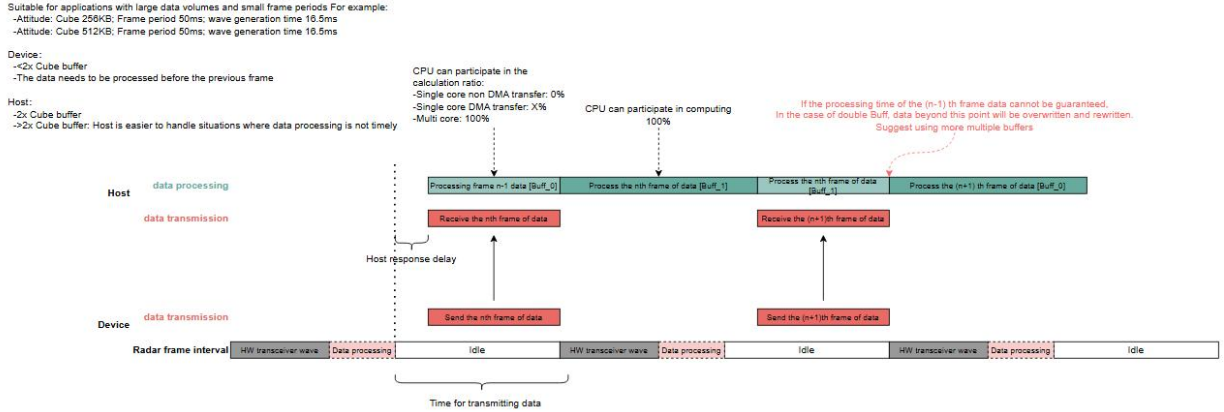


图 5-5 Data Cube 单帧上报多帧处理示意图

5.2.3 多帧上报和处理

若 Host 响应延时较大（如 Linux）或帧周期较小数据量较少的应用可累积多帧上报。Device 和 Host 需要更多的 buff 缓存，但传输时间可与“HW transceiver wave”和“Data processing”并行。

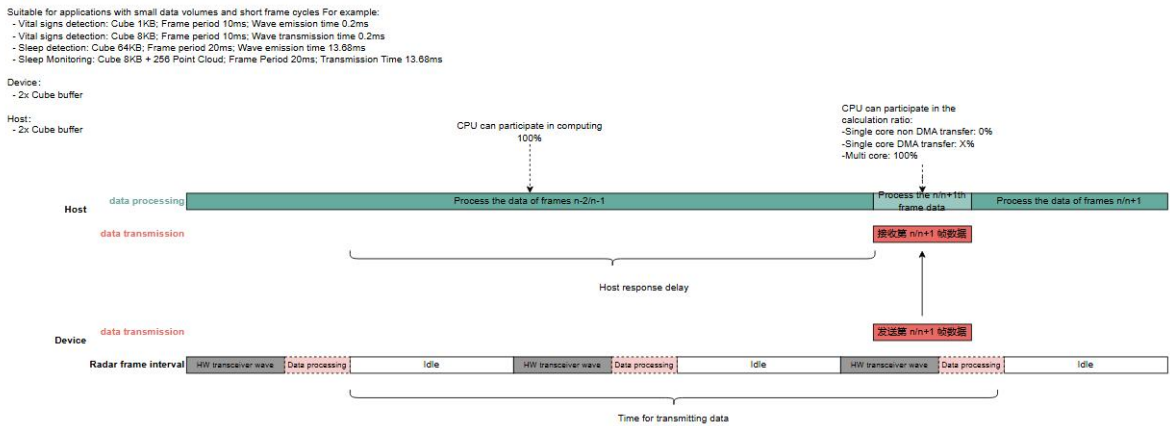


图 5-6 Data Cube 多帧上报处理示意图

5.2.4 其他信息上报

其他如用户处理结果或调试信息，其上报通常与帧周期不关联。任意时刻均可上报，时序图类似“点云多帧处理”。

6 附录-B Host 实时性参数

SPI/I2C 传输模式，影响传输效率的 Host 实时性参数包括以下 4 个。

表 6-1 Host 实时性参数

Item	Notes	RTOS	Linux
Response Latency	Report-IO 拉高至 Host 发送 Poll 包之间的延迟。对于 Data Cube 传输，影响第一包的传输，后续包级联不影响。与 host 响应 IO 性能有关。	50us~2ms	10ms ~ 20ms
poll-delay	Host 发送 poll 包至 Device 准备好数据包并被 Host 读数据包 Head 之间的时间。与 device 处理 poll 性能和 host 切换至读取性能有关。	500us	1ms
Head-delay	Device 的上报数据被 host 读取 head 后至读取 payload 之间的延迟。与 host 连续读取和计算 head checksum 性能有关。	50us	50us
Inter-packet	Host 读取完上报数据至读取下一包的头之间的延迟。与 host 连续读取和计算 payload checksum 性能有关。	100us	50us

对应时序图如下：



图 6-1 Host 实时性参数示意图

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。